

M05 Teaching Book

Python for Data Engineering - self-contained learner edition

HOW TO USE

This book can carry the lesson even if an external video changes or disappears. For assigned videos, watch the mapped scope first, close it, then learn independently from this page. Do not keep the tutorial open during the independent task.

ENGINEERING LOOP

Receipt -> profile -> validate/quarantine -> transform -> reconcile -> publish state -> observe/test -> explain. A script running without exception is not proof that the data is correct.

EVIDENCE STANDARD

Save the exact input identity/hash where required, commands/code, validation/reconciliation output, error/recovery evidence, and a short own-words explanation. Protected review is repair material, not first-pass teaching.

DE05-01 - CSV properly: parse, inspect, preserve raw

ASSIGNED VIDEO / SUPPORT ROUTE

VIDEO_PRIMARY: Corey Schafer - Reading and Writing CSV Files in Python

Open assigned source

Full focused CSV lesson. Watch first; close video before the Teaching Book.

1. STARTING STATE

Prerequisite: M04 completion. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Parse CSV deliberately, inspect its contract, and preserve raw input before transformation.

3. TEACH IT TO YOURSELF

A CSV file is text with a tabular convention, not a magically typed table. Preserve the received bytes or immutable source copy so later debugging can distinguish source from transformation.

4. WORKED DEMONSTRATION

Use `csv.DictReader` or `pandas read_csv` for inspection. Record filename, row count, columns, encoding assumption and a SHA-256/source-copy note before changing values.

5. GUIDED REPRODUCTION

Inspect `orders_good.csv`. Save raw row count, field names and first/last `order_id`. Copy it to an immutable `raw_received` folder without editing.

6. INDEPENDENT TASK

Given `orders_new.csv`, produce a source receipt note and prove the preserved raw copy is byte-identical to the supplied source.

7. VALIDATE

Row count and field list are explicit; raw copy hash equals source hash; no transformation overwrites raw.

8. COMMON FAILURE / RECOVER

Failure to avoid: Opening CSV in Excel and saving it back before preserving the original can silently change formatting/types. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Receive `data/retest/shipments_retest.csv`. Save its SHA-256 and row/field controls before doing any cleanup. Create your own byte-identical receipt copy under `data/work/retest_received/`.

10. EXPLAIN BACK

Without the book/video open, explain csv properly: parse, inspect, preserve raw in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-01 mastered.

DE05-02 - JSON and JSONL from zero

ASSIGNED VIDEO / SUPPORT ROUTE

VIDEO_PRIMARY: Corey Schafer - Working with JSON Data using the json Module

Open assigned source

Full focused JSON lesson. Teaching Book adds JSONL and line-level recovery.

1. STARTING STATE

Prerequisite: DE05-01. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Read and write JSON and JSONL while explaining their different record shapes.

3. TEACH IT TO YOURSELF

JSON can represent one nested value/document. JSONL stores one JSON value per line, useful for append/stream-style records and easy line-level recovery.

4. WORKED DEMONSTRATION

Read orders_sample.json with json.load; write a few records to .jsonl one record per line; read back line-by-line.

5. GUIDED REPRODUCTION

Convert six sample JSON records to JSONL and prove every line parses independently.

6. INDEPENDENT TASK

Create a new three-record nested JSON source, convert to JSONL, then reconstruct the list and compare IDs/counts.

7. VALIDATE

Every JSONL line parses; IDs/count reconcile; nested data meaning is not flattened accidentally.

8. COMMON FAILURE / RECOVER

Failure to avoid: Treating a JSON array file as if each physical line were a full JSON record. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Convert data/retest/shipments_sample.json into JSONL one object per line, then reconstruct the list. Prove shipment_id count/set is identical.

10. EXPLAIN BACK

Without the book/video open, explain json and jsonl from zero in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-02 mastered.

DE05-03 - Paths, folders and deterministic batch files

ASSIGNED VIDEO / SUPPORT ROUTE

OFFICIAL_REFERENCE: Python 3.14 pathlib documentation

Open assigned source

Basic use, Path construction/resolution, directory creation and file I/O. No weak tutorial video invented.

1. STARTING STATE

Prerequisite: DE05-02. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Use pathlib and deterministic folder/file naming so batch runs are reproducible.

3. TEACH IT TO YOURSELF

Paths are part of pipeline behavior. Derive paths from project root/config and name run outputs from explicit business/run keys, not random Downloads locations.

4. WORKED DEMONSTRATION

Use Path(__file__).resolve() and project-relative directories; create received/accepted/rejected/output/state folders; construct a deterministic batch filename.

5. GUIDED REPRODUCTION

Build batch_path(run_date, source_name) and show the same inputs produce the same path.

6. INDEPENDENT TASK

For a new source and run date, create the expected landing/output paths from config and prove rerunning path construction makes no duplicate/random folder.

7. VALIDATE

Same logical run keys produce same intended paths; no personal absolute path is required.

8. COMMON FAILURE / RECOVER

Failure to avoid: Hard-coding C:/Users/.../Downloads makes the pipeline non-portable. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

For source=shipments and run_date=2026-08-15, derive landing/accepted/rejected paths twice from project root/config. Prove identical inputs produce identical paths and no C:/Users/... dependency appears.

10. EXPLAIN BACK

Without the book/video open, explain paths, folders and deterministic batch files in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-03 mastered.

DE05-04 - pandas profiling before transformation

ASSIGNED VIDEO / SUPPORT ROUTE

VIDEO_SUPPORT: Luke Barousse - Python for Data Analytics

Open assigned source

4:22:41-4:47:03 - pandas introduction and inspection.

1. STARTING STATE

Prerequisite: DE05-03. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Profile a pandas batch before transforming it.

3. TEACH IT TO YOURSELF

Profiling answers what arrived: rows, columns, types, missingness, candidate-key uniqueness and simple ranges. Transformation should start only after these controls are recorded.

4. WORKED DEMONSTRATION

Load orders_good.csv, inspect shape/dtypes/nulls/duplicates and basic numeric/date ranges before adding business logic.

5. GUIDED REPRODUCTION

Create a profiling dictionary and save it as JSON evidence.

6. INDEPENDENT TASK

Profile orders_profile_variant.csv and flag at least three suspicious properties without fixing them yet.

7. VALIDATE

Profile is based on observed source; row/key/null/type controls are saved separately from cleaning.

8. COMMON FAILURE / RECOVER

Failure to avoid: Cleaning while profiling destroys evidence of what actually arrived. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Profile shipments_retest.csv before cleaning. Save rows, columns, dtypes, missing counts, duplicate shipment_id count and suspicious date/value/quantity observations.

10. EXPLAIN BACK

Without the book/video open, explain pandas profiling before transformation in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-04 mastered.

DE05-05 - Cleaning with pandas and rejected rows

ASSIGNED VIDEO / SUPPORT ROUTE

VIDEO_SUPPORT: Luke Barousse - Python for Data Analytics

Open assigned source

4:47:03-5:08:08 - pandas cleaning and analysis.

1. STARTING STATE

Prerequisite: DE05-04. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Clean with pandas while preserving rejected rows and explicit reasons.

3. TEACH IT TO YOURSELF

A production-minded cleaning step should separate accepted and rejected data. Bad rows are evidence, not trash. Attach reason codes rather than silently dropping.

4. WORKED DEMONSTRATION

Normalize safe text, parse dates/numbers with explicit error handling, validate qty/price rules, and write accepted/rejected outputs.

5. GUIDED REPRODUCTION

Process orders_bad.csv so every invalid row gets one or more reason codes.

6. INDEPENDENT TASK

On a changed bad batch, produce accepted + rejected datasets and reconcile source_rows = accepted_rows + rejected_rows.

7. VALIDATE

All source rows accounted for; rejects retain source fields + reason; no bad row silently disappears.

8. COMMON FAILURE / RECOVER

Failure to avoid: dropna() or errors="coerce" followed by filtering can hide defects if rejects are not preserved. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Split shipments_retest.csv into accepted and rejected rows with explicit reason codes. Prove input_rows = accepted_rows + rejected_rows and retain original fields.

10. EXPLAIN BACK

Without the book/video open, explain cleaning with pandas and rejected rows in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-05 mastered.

DE05-06 - pandas merge, cardinality and aggregation

ASSIGNED VIDEO / SUPPORT ROUTE

VIDEO_SUPPORT: Luke Barousse - Python for Data Analytics

Open assigned source

7:15:11-7:34:04 - merge and concat. Teaching Book adds cardinality validation and fan-out controls.

1. STARTING STATE

Prerequisite: DE05-05. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Merge pandas tables with cardinality checks and aggregate without fan-out.

3. TEACH IT TO YOURSELF

Before merge, state left/right grain and prove uniqueness on the intended one side. `validate=` in pandas merge can enforce expected cardinality.

4. WORKED DEMONSTRATION

Merge orders to customers on `customer_id` with `validate="many_to_one"`; compare row count and unmatched keys; aggregate accepted sales by region.

5. GUIDED REPRODUCTION

Run the valid merge, then deliberately duplicate a customer key and observe the validation failure.

6. INDEPENDENT TASK

Use a changed dimension with one missing and one duplicated key; diagnose both without using `drop_duplicates` as a blind fix.

7. VALIDATE

Pre/post row counts preserved when expected; duplicate one-side key blocks the merge; unmatched count is explicit; grouped total reconciles.

8. COMMON FAILURE / RECOVER

Failure to avoid: A merge can run and still multiply money if the right-side key is not unique. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Attempt many-to-one merge of shipment rows to `carriers_retest.csv`. Detect the deliberate duplicate carrier key before trusting totals, then separately report unmatched carrier IDs. Do not hide either issue with `drop_duplicates()`.

10. EXPLAIN BACK

Without the book/video open, explain pandas merge, cardinality and aggregation in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-06 mastered.

DE05-07 - Data validation: record, batch and relationship rules

ASSIGNED VIDEO / SUPPORT ROUTE

VIDEO_PRIMARY: Corey Schafer - Complete Pydantic Data Validation Course

Open assigned source

03:39-21:46 basic models/errors; 31:49-49:05 field constraints; 49:06-1:00:09 custom validators; 1:00:10-1:11:57 nested models. Teaching Book adds batch and relationship rules.

1. STARTING STATE

Prerequisite: DE05-06. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Implement record, batch and relationship validation as explicit contracts.

3. TEACH IT TO YOURSELF

Validation has layers: record field/type/range rules; batch row/key/freshness rules; relationship rules against reference data. Collect actionable failures before publish.

4. WORKED DEMONSTRATION

Define an OrderRecord Pydantic model for record shape and add batch uniqueness + customer relationship checks outside the model.

5. GUIDED REPRODUCTION

Validate good data and a packaged bad batch. Produce reason-coded error output.

6. INDEPENDENT TASK

Add a new rule (allowed status or max future date) and prove a violating record is rejected while valid records still pass.

7. VALIDATE

Good data passes; each controlled defect is detected; validation report identifies row/key/rule; publish is blocked on critical failure.

8. COMMON FAILURE / RECOVER

Failure to avoid: Using type hints alone and assuming runtime values are validated automatically. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Create ShipmentRecord validation plus batch uniqueness and carrier relationship checks. Prove the packaged invalid date, zero units, negative unit value, missing carrier, duplicate shipment ID and invalid status are surfaced by named rules.

10. EXPLAIN BACK

Without the book/video open, explain data validation: record, batch and relationship rules in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-07 mastered.

DE05-08 - HTTP and REST APIs from zero

ASSIGNED VIDEO / SUPPORT ROUTE

VIDEO_PRIMARY: Corey Schafer - Python Requests Tutorial

Open assigned source

00:00-25:00 full focused Requests lesson for request/response, GET, parameters, JSON and authentication context. Use the bundled local API for deterministic practice.

SECONDARY SUPPORT - OPTIONAL

OFFICIAL_SECONDARY: MDN HTTP Overview - Request/response structure, methods and status-code classes [open]

1. STARTING STATE

Prerequisite: DE05-07. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Explain HTTP/REST request-response basics using a safe local training API.

3. TEACH IT TO YOURSELF

HTTP exchanges a request for a response. Method, URL/path, query parameters, headers, status and body are separate parts. 2xx/4xx/5xx categories guide handling.

4. WORKED DEMONSTRATION

Start mock_api_server.py. Inspect /health and /orders with curl/browser/requests. Record status, headers/body shape.

5. GUIDED REPRODUCTION

Call GET /health and GET /orders?page=1&per_page=4; explain path vs query parameter.

6. INDEPENDENT TASK

Call /missing and /orders?fail=1; classify each result and state whether pipeline should retry, stop or treat as valid.

7. VALIDATE

200 payload shape is understood; 404 is not parsed as valid orders; 500 is classified as server failure.

8. COMMON FAILURE / RECOVER

Failure to avoid: Calling response.json() and using the result before checking status/contract. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Start the bundled local API. Call /health, /shipments?page=1&per_page=3, /missing and /shipments?fail=1. Record method/path/query/status and classify continue/stop/retry behavior.

10. EXPLAIN BACK

Without the book/video open, explain http and rest apis from zero in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-08 mastered.

DE05-09 - Calling APIs safely with requests

ASSIGNED VIDEO / SUPPORT ROUTE

OFFICIAL_PRIMARY: Requests 2.34.2 Quickstart

Open assigned source

Make a Request; Passing Parameters; JSON Response Content; Response Status Codes; Timeouts; Errors and Exceptions. You already watched the visual foundation in DE05-08.

SECONDARY SUPPORT - OPTIONAL

OFFICIAL_SECONDARY: Requests Advanced Usage - Sessions and request behavior [open]

1. STARTING STATE

Prerequisite: DE05-08. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Call APIs safely with requests using timeout, status handling and response validation.

3. TEACH IT TO YOURSELF

Network calls can fail slowly or return invalid payloads. A safe client uses timeouts, status handling, schema checks and bounded useful logging.

4. WORKED DEMONSTRATION

requests.Session/get with timeout; raise_for_status; parse JSON; validate expected keys before returning records.

5. GUIDED REPRODUCTION

Implement fetch_page() for local API and test 200/404/500 behavior.

6. INDEPENDENT TASK

Point fetch_page at a changed endpoint response missing expected items; fail clearly rather than returning empty success.

7. VALIDATE

Timeout configured; non-2xx does not enter normal processing; payload contract failures are distinct from HTTP failures.

8. COMMON FAILURE / RECOVER

Failure to avoid: except Exception: return [] turns outages into false "zero records" success. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Implement a safe page fetch for /shipments with timeout, status check and required payload keys. Then call /shipments?bad_payload=1 and prove it fails as a payload-contract error rather than returning zero rows.

10. EXPLAIN BACK

Without the book/video open, explain calling apis safely with requests in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-09 mastered.

DE05-10 - Pagination and authentication without leaking secrets

ASSIGNED VIDEO / SUPPORT ROUTE

OFFICIAL_PRIMARY: Requests 2.34.2 Authentication

Open assigned source

Basic Authentication and custom authentication concepts. Deterministic pagination and secret hygiene are course-built against the bundled local API.

1. STARTING STATE

Prerequisite: DE05-09. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Handle pagination and authentication without leaking secrets.

3. TEACH IT TO YOURSELF

Pagination is state: page/cursor advances only after a successful validated response. Authentication belongs in environment/secret storage, never source code or screenshots.

4. WORKED DEMONSTRATION

Use API_TOKEN environment variable; send Authorization header; loop next_page until None; collect IDs and prove no duplicates/gaps.

5. GUIDED REPRODUCTION

Fetch all local API pages with TRAINING_TOKEN supplied via environment. Save page/count evidence, not the token.

6. INDEPENDENT TASK

Change per_page and repeat; total IDs must remain the same. Then test missing/invalid token and prove no secret is logged.

7. VALIDATE

All expected IDs fetched once; token absent from source/log; auth failure stops clearly; pagination termination is deterministic.

8. COMMON FAILURE / RECOVER

Failure to avoid: Hard-coding bearer token into Python or logging full Authorization headers. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Fetch all /shipments pages using API_TOKEN from the environment. Repeat with per_page=2 and per_page=4; the ordered unique shipment ID set must match. Prove the token is absent from code/log evidence.

10. EXPLAIN BACK

Without the book/video open, explain pagination and authentication without leaking secrets in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-10 mastered.

DE05-11 - Retries, rate limits and partial failure

ASSIGNED VIDEO / SUPPORT ROUTE

OFFICIAL_REFERENCE: Requests 2.34.2 Advanced Usage

Open assigned source

Sessions; Transport Adapters; Example: Automatic Retries; Timeouts. Retry policy is failure-injected locally in this course.

1. STARTING STATE

Prerequisite: DE05-10. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Design bounded retries for rate limits/transient failures and preserve partial-failure evidence.

3. TEACH IT TO YOURSELF

Retry only failures that may recover, bound attempts, honor Retry-After/backoff, and never retry permanent data-contract errors blindly.

4. WORKED DEMONSTRATION

Implement retry wrapper for 429/5xx with max attempts and small training backoff. Save attempt/status log. Do not retry 400/401/validation failures.

5. GUIDED REPRODUCTION

Use local ?rate_limit=1 and ?fail_once=1 training cases; observe bounded recovery/failure.

6. INDEPENDENT TASK

Simulate a page that fails permanently after earlier pages succeeded; preserve last safe page and do not falsely mark full batch complete.

7. VALIDATE

Retries bounded; 429/5xx policy explicit; partial completion state distinguishable from full success; no infinite loop.

8. COMMON FAILURE / RECOVER

Failure to avoid: Blind retry around all exceptions can hammer a service and conceal permanent bad data. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Use /shipments?rate_limit_once=1 to prove one bounded recovery, then /shipments?fail=1 to prove a permanent failure is not mislabeled full success. Save attempt count and last-safe state.

10. EXPLAIN BACK

Without the book/video open, explain retries, rate limits and partial failure in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-11 mastered.

DE05-12 - Python + PostgreSQL with SQLAlchemy

ASSIGNED VIDEO / SUPPORT ROUTE

OFFICIAL_PRIMARY: SQLAlchemy 2.0 Unified Tutorial

Open assigned source

Establishing Connectivity - the Engine; Working with Transactions and the DBAPI; Working with Data. No Docker execution in M05.

SECONDARY SUPPORT - OPTIONAL

OFFICIAL_OPTIONAL_VISUAL: SQLAlchemy Library - 2.0 tutorial by Mike Bayer - Optional visual for Engine/Connection/transaction concepts after the assigned 2.0 docs [open]

1. STARTING STATE

Prerequisite: DE05-11. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Use SQLAlchemy to connect Python to the already-installed PostgreSQL lab safely.

3. TEACH IT TO YOURSELF

SQLAlchemy manages engine/connection and parameterized execution; PostgreSQL was first installed/verified in M03. M05 teaches Python integration, not database modeling.

4. WORKED DEMONSTRATION

Read DATABASE_URL from environment; create_engine; run SELECT version()/simple staging table; insert accepted rows using parameterized SQL/SQLAlchemy Core; transaction rollback on controlled failure.

5. GUIDED REPRODUCTION

Load a small accepted batch into a dedicated m05 schema/table, then query row count and total; rerun with stable key/upsert or clean-reset strategy stated explicitly.

6. INDEPENDENT TASK

Load a changed batch using a separate run_id. Prove a failed transaction does not leave half-published rows.

7. VALIDATE

No credential in code; connection works; row/key/totals reconcile; failure rolls back; rerun policy is explicit.

8. COMMON FAILURE / RECOVER

Failure to avoid: Embedding postgres password in source or assuming commit happened because execute() returned. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Load the accepted retest shipment batch to a separate m05_retest table/run_id using SQLAlchemy and the existing M03 PostgreSQL install. Inject one transaction failure and prove rollback leaves no half-published batch.

10. EXPLAIN BACK

Without the book/video open, explain python + postgresql with sqlalchemy in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-12 mastered.

DE05-13 - Configuration, environment variables and secret hygiene

ASSIGNED VIDEO / SUPPORT ROUTE

VIDEO_PRIMARY: Corey Schafer - Python: Using Environment Variables

Open assigned source

Full focused Windows environment-variable lesson. Teaching Book adds config layering and redaction.

1. STARTING STATE

Prerequisite: DE05-12. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Separate configuration, environment-specific values and secrets from logic.

3. TEACH IT TO YOURSELF

Config is data controlling behavior. Non-secret defaults/examples can live in repo; secrets come from environment/secret manager and must never be committed.

4. WORKED DEMONSTRATION

Load config.example.json; override selected values with env vars; validate required settings; redact secrets in logs.

5. GUIDED REPRODUCTION

Override output directory and DATABASE_URL via environment without editing Python.

6. INDEPENDENT TASK

Create a second config profile with different batch size/output path; prove code unchanged and secret value never printed.

7. VALIDATE

Same code runs with two configs; missing required setting fails clearly; secrets are excluded from Git/log evidence.

8. COMMON FAILURE / RECOVER

Failure to avoid: A .env file is convenient locally but is not safe to commit just because it is hidden in Explorer. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Create a second non-secret config profile for shipments with a different output folder/batch size. Override the fake training token through the environment without editing pipeline logic; scan evidence for secret leakage.

10. EXPLAIN BACK

Without the book/video open, explain configuration, environment variables and secret hygiene in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-13 mastered.

DE05-14 - Logging, testing and reusable pipeline code

ASSIGNED VIDEO / SUPPORT ROUTE

VIDEO_PRIMARY: Corey Schafer - Logging Basics

Open assigned source

Full focused logging lesson. The Teaching Book also exposes the current pytest documentation as secondary support for regression tests.

SECONDARY SUPPORT - OPTIONAL

OFFICIAL_SECONDARY: pytest documentation - Writing/running tests and logging capture [open]

1. STARTING STATE

Prerequisite: DE05-13. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Combine structured logging, tests and reusable pipeline code around clear boundaries.

3. TEACH IT TO YOURSELF

Reliable code separates pure transformations from I/O, logs run context/counts, and tests both small functions and integrated behavior.

4. WORKED DEMONSTRATION

Refactor into read/validate/transform/write functions; add pytest unit tests and one temp-directory integration test; structured log contains run_id/source/counts/status.

5. GUIDED REPRODUCTION

Run tests, then inject one bad record and confirm the expected test/failure evidence.

6. INDEPENDENT TASK

Add one regression test for a bug you deliberately create and repair. Explain what the test prevents.

7. VALIDATE

Tests fail before repair and pass after; logs identify run and counts; functions have clear responsibilities; failures propagate.

8. COMMON FAILURE / RECOVER

Failure to avoid: A green test suite that never checks bad input/rerun behavior gives false confidence. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Write a regression test for one shipment validation/logging bug, capture the failing test first, repair code, then capture the passing test. Log run_id, source, accepted/rejected counts and final status.

10. EXPLAIN BACK

Without the book/video open, explain logging, testing and reusable pipeline code in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-14 mastered.

DE05-15 - Integrated Python data-engineering pipeline

ASSIGNED VIDEO / SUPPORT ROUTE

PRACTICE_ONLY: Independent integrated checkpoint - no tutorial video by design
Build from the saved learner attempt. External references stay closed until a genuine attempt exists.

1. STARTING STATE

Prerequisite: DE05-14. Open only the M05 learner project and this lesson's practice. Keep protected Review closed. Never use real credentials in course files/evidence.

2. OUTCOME

Build and defend an integrated Python ingestion pipeline with safe reruns and PostgreSQL/output evidence.

3. TEACH IT TO YOURSELF

The final module proof combines preserve raw -> profile -> validate/quarantine -> transform -> API/file ingestion -> safe state -> optional PostgreSQL load -> logs/tests -> reconciliation.

4. WORKED DEMONSTRATION

Use M05_PIPELINE_TODO.py as blank-slate scaffold. The lesson book explains architecture, not a finished copy-paste solution.

5. GUIDED REPRODUCTION

Build one file-source run and one API-source run. Save source receipt, accepted/rejected counts, output/database controls, log and rerun result.

6. INDEPENDENT TASK

Run a fresh changed batch with one duplicate, one invalid row and one transient API failure. Recover without corrupting previously published data.

7. VALIDATE

PASS only when all integrated pipeline contracts are evidenced: raw/source receipt preserved; accepted + rejected reconcile to input; no silent bad-data publish; rerun creates no duplicate publish; transient failure follows bounded recovery; permanent/partial failure is not labeled full success; no secret appears in code/log/evidence; output or PostgreSQL state reconciles; tests/logs support the run; and another learner could reproduce the run from README/config. Any unresolved critical contract => TARGETED REPAIR.

8. COMMON FAILURE / RECOVER

Failure to avoid: Treating "script exited 0" as sufficient without proving data correctness, state and recovery. Recovery: stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by dropping data or returning an empty result.

9. FRESH CHANGED RETRY IF ASSIGNED

Run the complete changed shipments pipeline from raw receipt + local API through validation/quarantine, transformation, bounded retry state, optional PostgreSQL load, logs/tests and reconciliation. Preserve the original practical attempt; this fresh run is separate evidence.

10. EXPLAIN BACK

Without the book/video open, explain integrated python data-engineering pipeline in 60-90 seconds: what one input row/response represents, the critical failure mode, the validation control, and what evidence proves success.

11. MASTER / FINISHED WHEN

Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and save evidence. A percentage score alone cannot mark DE05-15 mastered.